

Advance Jenkins Concepts

starts at 9:05pm

Agenda

1. Pipelines

1. Integrating Version Control

2. Plugins on the go

3. Types of Pipelines

1. Scripted Pipelines

2. Declarative Pipelines

3. Demo Declarative and Scripted Pipeline

2. Agent Management

1. Master and slave

2. Demo for adding a node and running a pipeline on agent.

3. Variables

4. Parameterized Jobs

1. Demo Checkbox Parameterized Pipeline

→ Pipelines

A **Jenkins Pipeline** is an automated sequence of steps that defines how code moves from development to production.

It allows us to **automate build, test, and deployment** processes in a structured way.

→ Car Factory

Assemble diff parts → Painting → Quality checks.



Final shipment

Build → Testing → Deployed.

- ① Check out → Build → Test → Deploy
- ② Build → Test → Deploy.

Build
Test
Deploy

Jenkinsfile

→ Git integration of a Public Repo in Jenkins

stages → stage → step
stage 2
stage 3

```
pipeline {  
    agent any // Runs on any available Jenkins agent  
  
    stages {  
        stage('Clone Repository') { // First stage: Cloning the repo  
            steps {  
                git branch: 'jenkins', url: 'https://github.com/vedantsharmascaler/testing_repo.git' // Cloning the repo  
            }  
        }  
  
        stage('Run Script') { // Second stage: Executing the script
```

```
steps {
```

```
    sh 'chmod +x script.sh' // Make the script executable
```

```
    sh './script.sh' // Execute the script
```

```
}
```

```
}
```

```
}
```

```
}
```

```
echo "echo Hello from Jenkins Pipeline!" > script.sh
```

```
chmod +x script.sh
```

→ Types of Jenkins Pipeline

- **Declarative Pipeline** (modern, simpler)
- **Scripted Pipeline** (more flexible, but complex)

→ Declarative

- Introduced in Jenkins 2.x as a simpler way to write pipelines.
- Uses a **structured syntax** with predefined keywords.
- Easier to read, write, and maintain.
- Enforces a **single `pipeline {}` block** with predefined stages.
- Fails fast due to built-in validation.

```
pipeline {  
    agent any // Runs on any available agent/node
```

```
    stages {
```

```
        stage('Build') {
```

```
            steps {
```

```
                echo 'Building the application...'
```

```
            }
```

```
        }
```

```
        stage('Test') {
```

```
            steps {
```

```
                echo 'Running tests...'
```

```
            }
```

```
        }
```

```
        stage('Deploy') {
```

```
            steps {
```

```
                echo 'Deploying application...'
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

→ Scripted Pipeline

- **Older and more flexible** but complex.
- More powerful but requires **deep Groovy knowledge**.
- Uses multiple `node {}` blocks to define where things run.

```
node {  
  
    stage('Build') {  
  
        echo 'Building the application...'  
  
    }  
  
  
    stage('Test') {  
  
        echo 'Running tests...'  
  
    }  
  
  
    stage('Deploy') {  
  
        echo 'Deploying application...'  
  
    }  
  
}
```

Try it

Some code

{

exception of
{
failure?

Feature	Declarative	Scripted
Structure	Enforces a strict structure (pipeline {})	More flexible, no enforced structure
Simplicity	Easier to read and write	More complex, requires Groovy scripting
Maintainability	Easier to maintain	Harder to manage for complex workflows
When to Use?	Simple to medium CI/CD pipelines	Advanced scenarios needing more control
Error Handling	Uses <code>post</code> blocks (<code>failure</code> , <code>always</code>).	Uses <code>try-catch-finally</code> .
Conditional Execution	Uses <code>when</code> conditions.	Uses <code>if</code> statements.
Readability	More readable and maintainable.	Less readable for beginners.

→ Scripted

✓ { Stage 1 → 5 mins
Stage 2 → 10 mins
Stage 3 → 25 mins
Stage 4 → 10 mins. → Failure?

→ Declarative . (Save time)
→ validates the syntax.

Jenkinsfile-declarative

```
pipeline {
```

```
  agent any
```

```
  options {
```

```
    preserveStashes() // Enables restarting from a failed stage
```

```
  }
```

```
  stages {
```

```
    stage('Clone Repository') {
```

```
      steps {
```

```
        echo 'Cloning repository...'
```

```
        git branch: 'jenkins', url: 'https://github.com/vedantsharmascaler/testing_repo.git'
```

```
      }
```

```
    }
```

```
  stage('Build') {
```



```
steps {
```

```
    echo 'Building application...'
```

```
}
```

```
}
```

```
stage('Syntax Error Demo') {
```

```
    stepa {
```

```
        echo 'Intentional syntax error'
```

```
    }
```

```
}
```

```
stage('Test') {
```

```
    steps {
```

```
        echo 'Running tests...'
```

```
    }
```

```
}
```

```
}
```

```
}
```

```
jenkinsfile-scripted
```

```
node {
```

```
    stage('Clone Repository') {
```

```
echo 'Cloning repository...'
```

```
git branch: 'jenkins', url: 'https://github.com/vedantsharmascaler/testing_repo.git'
```

```
}
```

```
stage('Build') {
```

```
echo 'Building application...'
```

```
}
```

```
stage('Syntax Error Demo') {
```

```
echo 'Intentional syntax error'
```

```
}
```

```
stage('Test') {
```

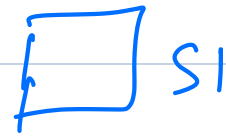
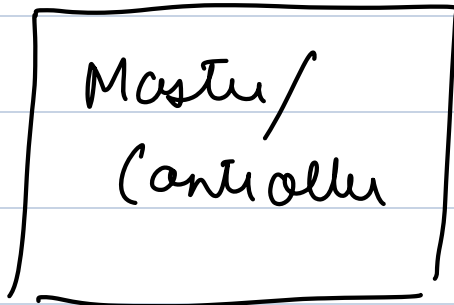
```
echo 'Running tests...'
```

```
}
```

```
}
```

Break → 10:30pm

→ Agent Management

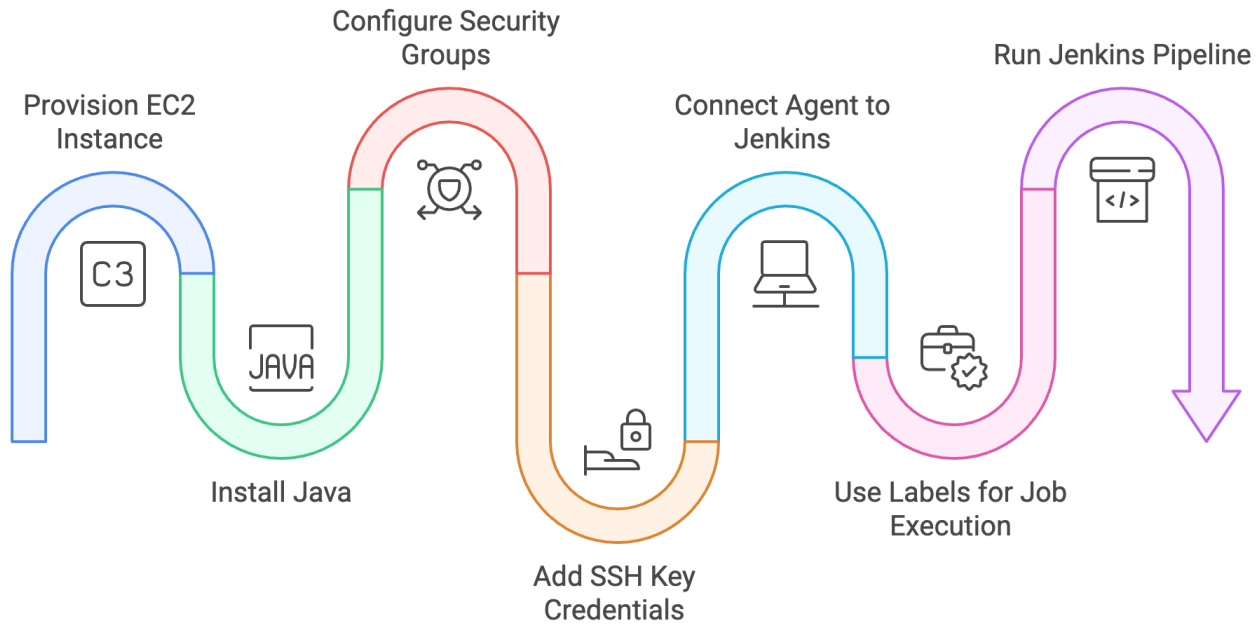


Labels → agents.

→ Advantages

- ① Distributed workload
- ② specialized Environment.
- ③ Isolation.

Setting Up EC2 Instance as Jenkins Agent



Step 1 → Provision EC2 machine.

Strategy	Description	Use Case
Manually Trusted Key Verification Strategy (Recommended)	Jenkins will prompt for manual approval when an agent first connects. Future connections are trusted.	Best for secure environments where you want to manually verify SSH host keys before accepting them.
Known Hosts File Verification Strategy	Uses the standard <code>~/.ssh/known_hosts</code> file for SSH key verification. If the key is missing or changed, the connection fails.	Best for pre-configured environments where SSH keys are already managed (e.g., via Ansible, Puppet, or Chef).
Accept First Connection Verification Strategy	Accepts the first connection without verification, but future key changes will cause failures.	Suitable for non-production environments where initial security is not a concern.
No Host Key Verification Strategy	Disables host key verification completely, allowing connections without authentication.	Only for testing environments where security is not a concern. Not recommended for production.

```
pipeline {  
  
    agent { label 'ec2-agent' } // Ensure job runs on EC2 agent  
  
    stages {  
  
        stage('Build') {  
  
            steps {  
  
                echo 'Building project...'  
  
                sh 'sleep 10'  
  
            }  
  
        }  
  
        stage('Test') {  
  
            steps {  
  
                echo 'Running tests...'  
  
                sh 'echo "Tests Passed!"'  
  
            }  
  
        }  
  
        stage('Deploy') {  
  
            steps {  
  
                echo 'Deploying application...'  
  
                sh 'echo "Deployment Complete"'  
  
            }  
  
        }  
  
    }  
}
```

}

}

→ Variables

① Build in env variables.

- `BUILD\_NUMBER`: The current build number (integer).

- `BUILD\_ID`: The unique identifier for the current build (timestamp).

- `BUILD\_URL`: URL to access the details of the current build.

- `JOB\_NAME`: The name of the job.

→ IP : 8080/env-vars.html

② Custom environment variables.

pipeline {

agent any

environment {

MY_VAR = 'Hello, World!'

ENVIRONMENT = 'production'

}

stages {

```
stage('Build') {
```

```
  steps {
```

```
    echo "Environment: $ENVIRONMENT"
```

```
    echo "Custom message: $MY_VAR"
```

```
  }
```

```
}
```

→ \$BUILD_NUMBER

```
}
```

```
}
```

→ Build Parameters

```
pipeline {
```

```
  agent any
```

```
  parameters {
```

```
    string(name: 'ENVIRONMENT', defaultValue: 'dev', description: 'Deployment environment')
```

```
    booleanParam(name: 'DEPLOY', defaultValue: true, description: 'Should we deploy?')
```

```
  }
```

```
  stages {
```

```
    stage('Build') {
```

```
      steps {
```

```
echo "Building for environment: ${params.ENVIRONMENT}"
```

```
echo "Should deploy? ${params.DEPLOY}"
```

```
}
```

```
}
```

```
}
```

```
}
```

④ Global variables

1. Go to **Manage Jenkins** → **Configure System**.

2. Under the **Global Properties** section, check the box **Environment Variables**.

3. Add key-value pairs for the global environment variables.

→ Parametrised Job

→ choice (choose one)

dev, prod, staging.

→ extended.



→ multiselect

→ Create a Checkbox Parameterised Pipeline.

① Install Plugin
→ Checkbox Parameter Plugin.

Build, Test, Deploy.

→ ['Build', 'Test', 'Deploy']